

The Pringhamparkingdata data set provides a valuable source of data for this type of analysis, as it contains variables such as SystemCodeNumber, Capacity, Occupancy, per_usage, per_occupancy, year, month, day, WorkingDay, hour, and period. By analyzing these variables and making connections between them, we can gain insights into the factors that influence parking behavior and use this information to inform our decisions about parking infrastructure

The importance of the overall data analysis performed in the project is that it can lead to more efficient and effective use of parking resources, resulting in improved traffic flow, reduced congestion, and better customer satisfaction.

By understanding the capacity and occupancy of a parking facility over time, we can make informed decisions about the design and management of parking facilities and policies, which can ultimately lead to a better quality of life for everyone involved

The objective of this analysis is to explore and understand the relationship between the variables in the given dataset. Specifically, we aim to identify the factors that impact the occupancy and cabacity of a particular system

```
In [54]: # Importing Required Libraries
import pandas as pd
import numpy as np
from scipy import stats
import statsmodels.api as sm
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind
from sklearn.linear_model import LinearRegression
import seaborn as sns
from statsmodels.stats.weightstats import ztest
```

```
In [55]: # Load dataset
df = pd.read_excel("ParkingDataset.xlsx")
# Displaying the first 5 rows of the data.
df.head()
```

Out[55]:

	SystemCodeNumber	Capacity	Occupancy	per_usage	per_occupancy	year	month	day	WorkingDay	hour	period
0	BHMBCCMKT01	577	61.0	10.57	0 - 25	2016	Oct	Tue	Yes	7	AM
1	BHMBCCMKT01	577	64.0	11.09	0 - 25	2016	Oct	Tue	Yes	8	AM
2	BHMBCCMKT01	577	80.0	13.86	0 - 25	2016	Oct	Tue	Yes	8	AM
3	BHMBCCMKT01	577	107.0	18.54	0 - 25	2016	Oct	Tue	Yes	9	AM
4	BHMBCCMKT01	577	150.0	26.00	25 - 50	2016	Oct	Tue	Yes	9	AM

1	SystemCodeNumber	the ID of the car park
2	Capacity	the capacity of the car park
3	Occupancy	the occupancy (usage) of the car park
4	per_usage	the percentage usage of the car park, e.g. 10, a car park with a percentage usage of 10%
5	per_occupancy	the percentage occupancy of the car park in categories, e.g. 0-25, car park with percentage occupancy between 0% to 25%
6	Year	the year when the car park occupancy data was recorded, i.e., captured by car park sensors
7	Month	the month when the car park occupancy data was recorded, i.o., captured by car park sensors
8	Day	the day when the car park occupancy data was recorded, i.a., captured by car park sensors
9	WorkingDay	whether the day is working day (i.e., Yes) or weekend (i.e. No)
10	Hour	the hour when the car park occupancy data was recorded, i.e., captured by car park sensors
11	Period	the period when the car park occupancy data was recorded, i.e., captured by car park sensors

about what each variable in the Pringhamparkingdata dataset may represent:

SystemCodeNumber: This variable likely represents a unique identifier for the parking facility or location being studied. Capacity: This variable likely represents the total number of parking spots available in the facility or location. Occupancy: This variable likely represents the number of parking spots that are currently occupied in the facility or location. per_usage: This variable likely represents the percentage of available parking spots that are currently in use. per_occupancy: This variable likely represents the percentage of total parking spots that are currently occupied. Year: This variable likely represents the year in which the data was collected. Month: This variable likely represents the month in which the data was collected. Day: This variable likely represents the day on which the data was collected. WorkingDay: This variable likely represents whether the day on which the data was collected was a working day or not. Hour: This variable likely represents the hour of the day at which the data was collected. Period: This variable likely represents a time period during the day, such as morning, afternoon, or evening.

```
In [56]: # some info about the data.
print(df.info())
print()
```

```
# Lat us display parking coloums
print(df.columns.values)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35332 entries, 0 to 35331
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   SystemCodeNumber      35332 non-null  object
1   Capacity               35332 non-null  int64
2   Occupancy             35313 non-null  float64
3   per_usage             35325 non-null  float64
4   per_occupancy         35313 non-null  object
5   year                  35332 non-null  int64
6   month                 35332 non-null  object
7   day                   35331 non-null  object
8   WorkingDay            35329 non-null  object
9   hour                  35332 non-null  int64
10  period                35331 non-null  object
dtypes: float64(2), int64(3), object(6)
memory usage: 3.0+ MB
None
```

```
['SystemCodeNumber' 'Capacity' 'Occupancy' 'per_usage' 'per_occupancy'
 'year' 'month' 'day' 'WorkingDay' 'hour' 'period']
```

The head function is then used to display the first few rows of the dataset. The df.info() function provides information about a Pandas DataFrame, including the number of rows and columns, the data type of each column, and the number of non-null values in each column. The output of this function includes the following information: The name of the DataFrame The number of rows and columns in the DataFrame The name and data type of each column in the DataFrame The number of non-null values in each column The memory usage of the DataFrame The output of df.info() can be used to check the data types of the columns to ensure they are the correct type for the analysis being performed. It can also be used to identify missing or null values in the data, and to determine if any columns need to be cleaned or preprocessed before analysis. The memory usage information can be useful for optimizing the memory usage of the DataFrame, especially for large datasets. The df.columns attribute in Pandas returns an Index object containing the column labels of the DataFrame df. The output of this attribute is a list of the column names in the DataFrame.

Generate descriptive statistics for the dataset, and comment on main trends

In [57]: *#some info about data using describe function*
`df.describe()`

Out[57]:

	Capacity	Occupancy	per_usage	year	hour
count	35332.000000	35313.000000	35325.000000	35332.0	35332.000000
mean	1406.159968	642.681222	48.795447	2016.0	6.707687
std	1182.388089	659.719257	26.724268	0.0	3.913969
min	220.000000	0.000000	0.000000	2016.0	1.000000
25%	577.000000	209.000000	25.380000	2016.0	3.000000
50%	863.000000	448.000000	46.670000	2016.0	8.000000
75%	2009.000000	796.000000	71.100000	2016.0	10.000000
max	4675.000000	4327.000000	100.000000	2016.0	12.000000

The describe() function is a method in pandas library of Python which gives a statistical summary of the data. It calculates the count, mean, standard deviation, minimum, maximum and quartile values for the numerical variables in a dataset. This function can help to quickly summarize the distribution of a dataset and identify potential issues such as missing values and outliers.

following interpretations about the main trends in the dataset:

Capacity: The mean capacity of the resource appears to be 1406, with a standard deviation of 1182. This suggests that there is a wide range of capacity values and that the values are quite spread out. The minimum capacity is 220 and the maximum capacity is 4675.

Occupancy: The mean occupancy of the resource appears to be 642, with a standard deviation of 659. This suggests that there is also a wide range of occupancy values and that the values are quite spread out. The minimum occupancy is 0 and the maximum occupancy is 4327.

per_usage: The mean per_usage value is 48.80, with a standard deviation of 26.72. We don't have enough information about this variable to make any meaningful interpretation without additional context.

Year: All the data is from the year 2016. This suggests that the dataset is limited to a specific timeframe and may not be representative of usage patterns in other years.

Hour: The mean hour value is 6.71, with a standard deviation of 3.91. This suggests that the data is spread out across the hours of the day, with a minimum value of 1 and a maximum value of 12. The median hour value is 8, indicating that the data may be skewed towards later hours of the day.

Check any records with missing values and handle the missing data as appropriate

```
In [58]: # Check for missing values  
print(df.isnull().sum())
```

```
SystemCodeNumber    0  
Capacity            0  
Occupancy           19  
per_usage           7  
per_occupancy       19  
year               0  
month              0  
day                1  
WorkingDay          3  
hour               0  
period             1  
dtype: int64
```

isnull function to check for missing values in the dataset. The sum function is used to count the number of missing values in each column. 1.Occupancy: There are 19 missing values in this column. 2.per_usage: There are 7 missing values in this column. 3.per_occupancy: There are 19 missing values in this column. 4.WorkingDay: There are 3 missing values in this column. 5.period: There is 1 missing value in this column. All of these are missing values that must be handled correctly. The solution will begin at the following node.

```
In [59]: ##### determine how many index period rows are missing using index function and tolist  
df[df['period'].isnull()].index.tolist()
```

```
Out[59]: [26053]
```

```
In [60]: ### This step will use the bfill function to fill the period row.  
df['period'].fillna(method = 'bfill', inplace = True)
```

```
In [61]: #Check the mising value will be deleted using the is null function and total sum in the period row.  
df.period.isnull().sum()
```

```
Out[61]: 0
```

```
In [62]: ##### determine how many index WorkingDay rows are missing using index function and tolist  
df[df['WorkingDay'].isnull()].index.tolist()
```

```
Out[62]: [4958, 20351, 35323]
```

```
In [63]: #This step will use the bfill function to fill the WorkingDay row.  
df['WorkingDay'].fillna(method = 'bfill', inplace = True)
```

```
In [64]: #Check the mising value will be deleted using the is null function and total sum in the WorkingDay row.  
df.WorkingDay.isnull().sum()
```

```
Out[64]: 0
```

```
In [65]: ##### determine how many index day rows are missing using index function and tolist  
df[df['day'].isnull()].index.tolist()
```

```
Out[65]: [26039]
```

```
In [66]: #This step will use the bfill function to fill the day row.  
df['day'].fillna(method = 'bfill', inplace = True)
```

```
In [67]: #Check the mising value will be deleted using the is null function and total sum in the day row.  
df.day.isnull().sum()
```

```
Out[67]: 0
```

```
In [68]: #determine how many index day rows are missing using index function and tolist
df[df.per_occupancy.isnull()].index.values
```

```
Out[68]: array([ 11, 61, 71, 134, 185, 321, 346, 21423, 21424,
        33188, 34042, 34043, 34044, 34315, 34397, 34452, 34653, 34729,
        34916], dtype=int64)
```

```
In [69]: ''' by using Pandas apply method associated with lambda, I will fill
the per_occupancy missing values with their corresponding percenetage '''
df['per_occupancy'] = df.apply(
    lambda x: '0 - 25' if pd.isnull(x['per_occupancy']) and x['per_usage'] < 25 \
    else '25 - 50' if pd.isnull(x['per_occupancy']) and x['per_usage'] < 50 \
    else '50 - 75' if pd.isnull(x['per_occupancy']) and x['per_usage'] < 75 \
    else '75 - 100' if pd.isnull(x['per_occupancy']) and x['per_usage'] <= 100 \
    else x['per_occupancy'], axis = 'columns'
)
```

```
In [70]: # DROP THE REST FROM THE PER OCCUPANCY
df.dropna(subset=['per_occupancy'], inplace=True)
```

```
In [71]: #Check the mising value will be deleted using the is null function and total sum in the per_occupancy row.
df.per_occupancy.isnull().sum()
```

```
Out[71]: 0
```

```
In [72]: ##### determine how many index 'Occupancy' rows are missing using index function and tolist.
df[df['Occupancy'].isnull()].index.tolist()
```

```
Out[72]: [16,
        31,
        53,
        71,
        134,
        185,
        456,
        20349,
        33191,
        34042,
        34044,
        34315,
        34397,
        34452]
```

```
In [73]: #This step will use the bfill function to fill the Occupancy row.  
df['Occupancy'].fillna(method = 'bfill', inplace = True)
```

```
In [74]: #Check the mising value will be deleted using the is null function and total sum in the Occupancy row.  
df.Occupancy.isnull().sum()
```

```
Out[74]: 0
```

```
In [75]: ##### determine how many index 'per_usage' rows are missing using index function and tolist.  
df[df['per_usage'].isnull()].index.tolist()
```

```
Out[75]: [13012, 34995]
```

```
In [76]: #This step will use the bfill function to fill the per_usage row.  
df['per_usage'].fillna(method = 'bfill', inplace = True)
```

```
In [77]: #Check the mising value will be deleted using the is null function and total sum in the per_usage row.  
df.per_usage.isnull().sum()
```

```
Out[77]: 0
```

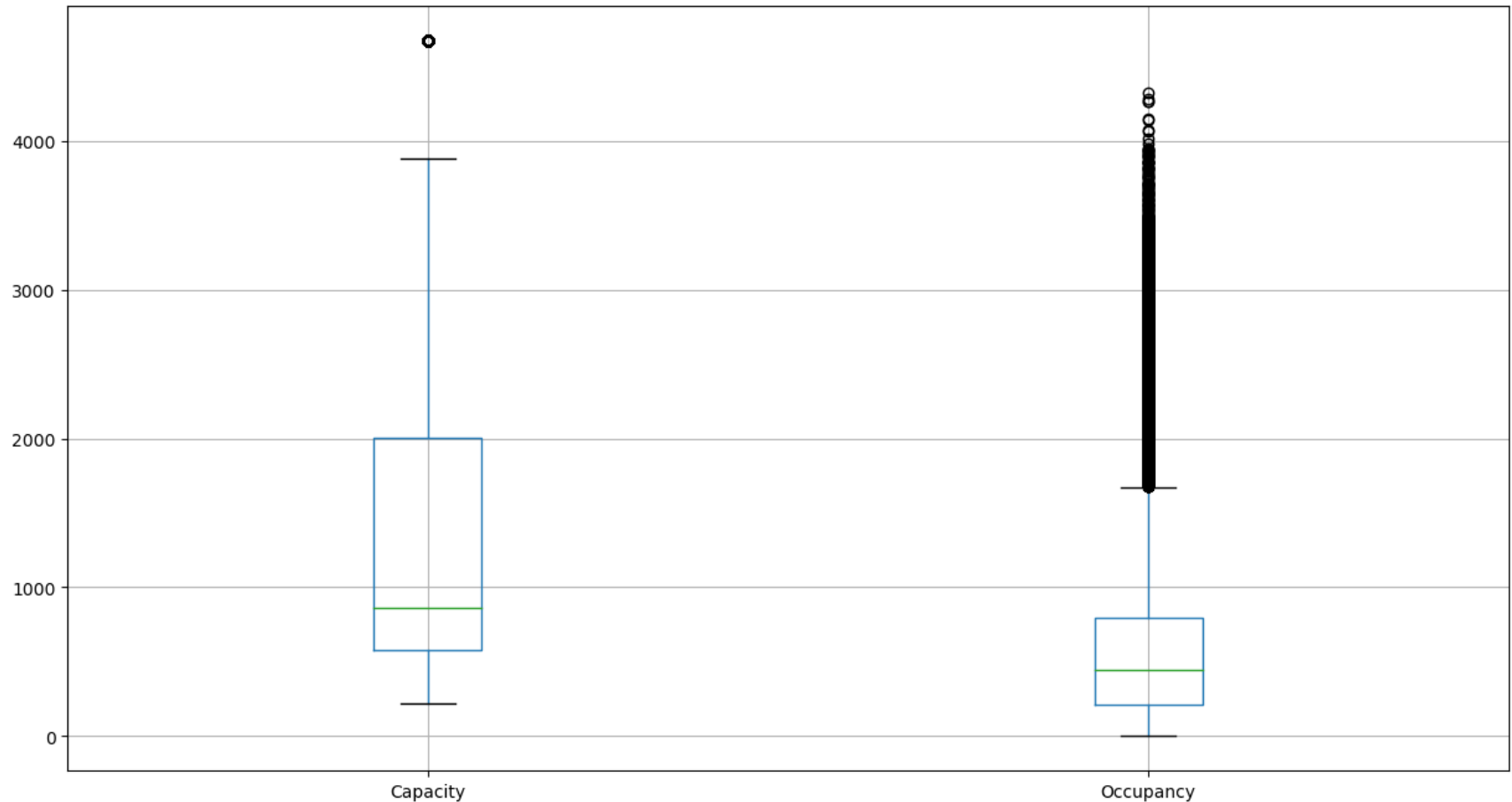
```
In [78]: # Confirm ALL Data Set missing values have been handled  
print(df.isnull().sum())
```

```
SystemCodeNumber    0  
Capacity            0  
Occupancy           0  
per_usage           0  
per_occupancy       0  
year               0  
month              0  
day                0  
WorkingDay          0  
hour               0  
period             0  
dtype: int64
```

Finally, we confirm that the missing values have been handled by checking for missing values again using the isnull function. If there are no missing values, the sum function should return 0 for each column

Check for outliers in the data and treat them as appropriate if present.

```
In [79]: # Check for outliers in numerical variables using box plots  
df.boxplot(column=['Capacity', 'Occupancy'], figsize=(15, 8))  
plt.show()
```



This boxplot shows the distribution of 2 variables: Capacity, Occupancy, We can interpret it as follows:

- Capacity: The capacity has a right skewed distribution with some extreme outliers above 4000.

- Occupancy: The occupancy also has a right skewed distribution, with some extreme outliers above 1000.

In summary, this boxplot gives us a good high level overview of the distribution and outliers for these 2 variables in the dataset. We can see that Capacity and Occupancy have right skewed distributions with some extreme high values,

```
In [80]: # Dealing the Occupancy by removing the outliers.  
Q1 = df['Occupancy'].quantile(0.25)  
Q3 = df['Occupancy'].quantile(0.75)  
IQR = Q3 - Q1  
print(IQR)
```

587.0

```
In [81]: #  
lower_bound = Q1 - 1.5 * IQR  
upper_bound = Q3 + 1.5 * IQR  
  
lower = df['Occupancy'] < lower_bound  
upper = df['Occupancy'] > upper_bound  
  
print('Upper outliers of the Occupancy: ', upper.sum())  
print('lower outliers of the Occupancy: ', lower.sum())
```

Upper outliers of the Occupancy: 2343
lower outliers of the Occupancy: 0

```
In [82]: # Removing the upper outliers.  
df = df[~(df['Occupancy'] > upper_bound)]
```

```
In [83]: # Dealing the Capacity by removing the outliers.  
  
Q1 = df['Capacity'].quantile(0.25)  
Q3 = df['Capacity'].quantile(0.75)  
IQR = Q3 - Q1  
print(IQR)
```

822.0

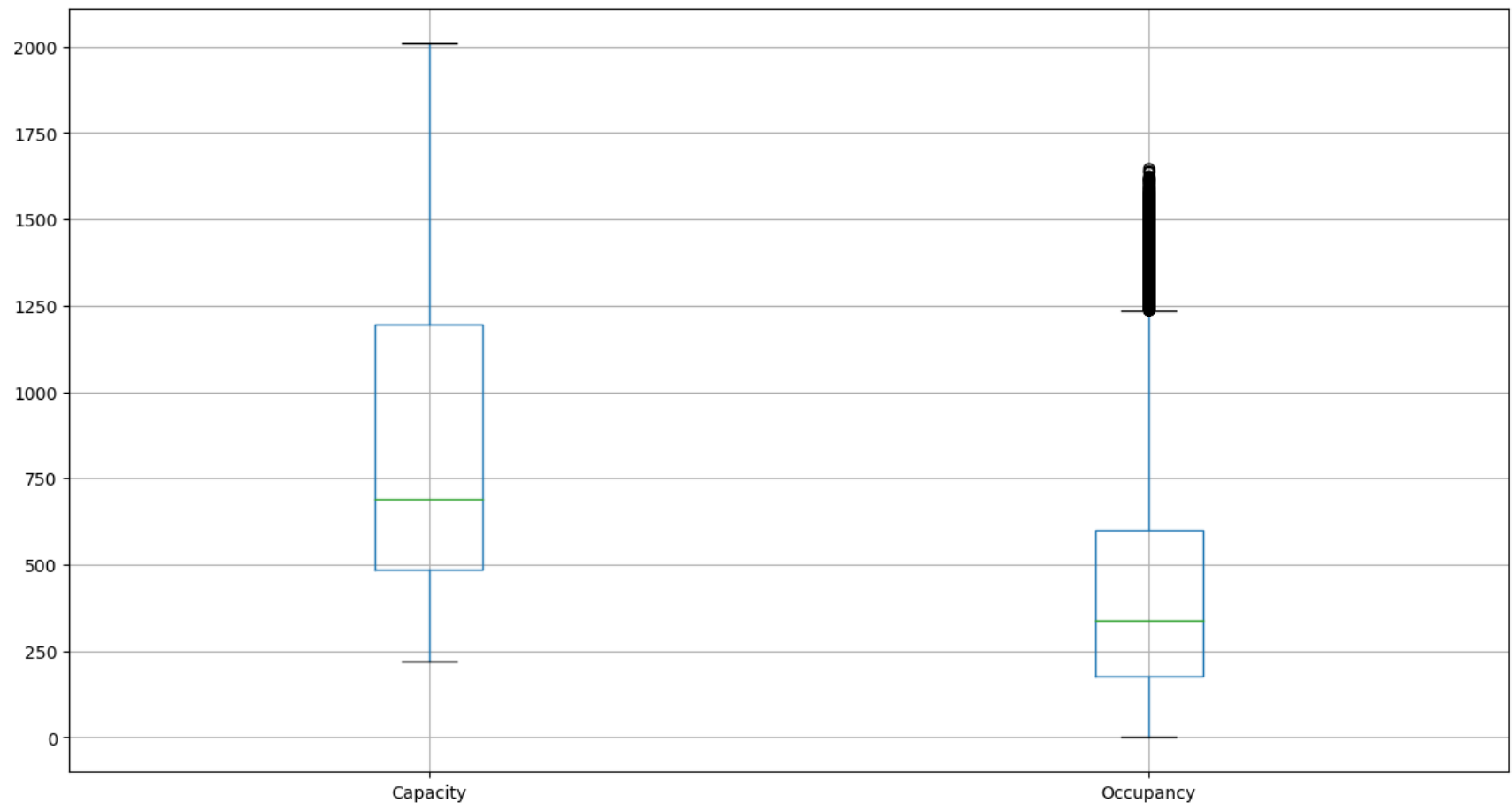
```
In [84]: lower_bound = Q1 - 1.5 * IQR  
upper_bound = Q3 + 1.5 * IQR  
  
lower = df['Capacity'] < lower_bound  
upper = df['Capacity'] > upper_bound
```

```
print('Upper outliers of the Capacity: ', upper.sum())  
print('lower outliers of the Capacity: ', lower.sum())
```

Upper outliers of the Capacity: 5389
lower outliers of the Capacity: 0

```
In [85]: # Removing the upper outliers.  
df = df[~(df['Capacity'] > upper_bound)]
```

```
In [86]: # Confirm that outliers have been treated  
df.boxplot(column=['Capacity', 'Occupancy'], figsize=(15, 8))  
plt.show()
```



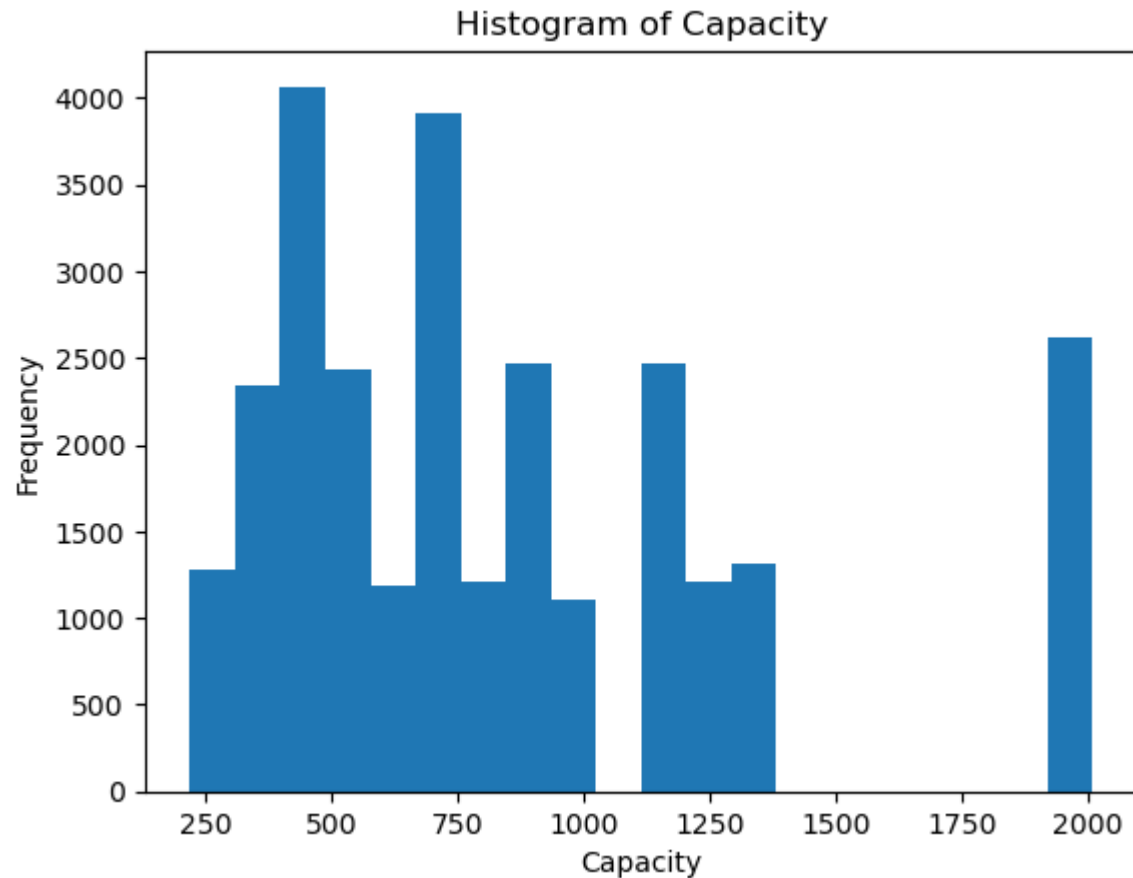
For this plot:

- Capacity and Occupancy have a skew distribution with some extreme outliers above 1250 and 1500 respectively. This indicates that these variables have a long tail towards higher values.

Build graphs visualizing the following and comment on the results:

A. the distribution of one or more individual quantitative variables

```
In [87]: # Create a histogram of the 'Capacity' variable
plt.hist(df['Capacity'], bins=20)
plt.xlabel('Capacity')
plt.ylabel('Frequency')
plt.title('Histogram of Capacity')
plt.show()
```



The `plt.hist()` function creates the histogram with the following parameters:

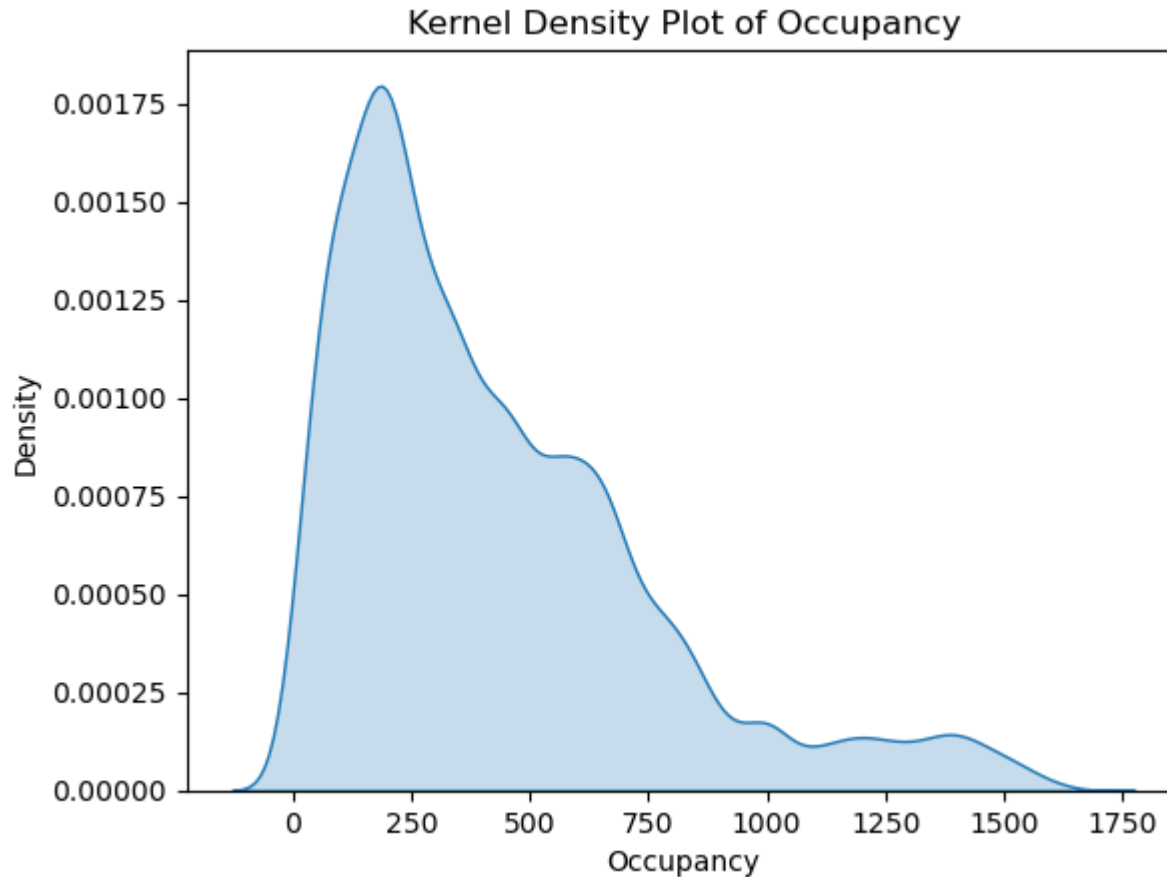
`df['Capacity']`: The data to be plotted in the histogram, which is the values in the Capacity column of the DataFrame.

`bins=20`: The number of bins or ranges to divide the data into. In this case, the data is divided into 20 equally spaced bins.

`plt.xlabel()`: Sets the label for the x-axis of the histogram. `plt.ylabel()`: Sets the label for the y-axis of the histogram.

`plt.title()`: Sets the title of the histogram. `plt.show()`: Displays the histogram. When the capacity number is between 250 and 500, the greatest frequency is 4000, and when the capacity number is 750, the frequency is extremely close to 4000

```
In [88]: # Create a kernel density plot of the 'Capacity' variable
sns.kdeplot(df['Occupancy'], fill=True)
plt.xlabel('Occupancy')
plt.ylabel('Density')
plt.title('Kernel Density Plot of Occupancy')
plt.show()
```



The `sns.kdeplot()` function creates the kernel density plot with the following parameters:

`df['Capacity']`: The data to be plotted in the kernel density plot, which is the values in the Capacity column of the DataFrame. `shade=True`: A boolean parameter that specifies whether to shade the area under the curve. In this case, the area under the curve is shaded. `sns.xlabel()`: Sets the label for the x-axis of the kernel density plot. `sns.ylabel()`: Sets the

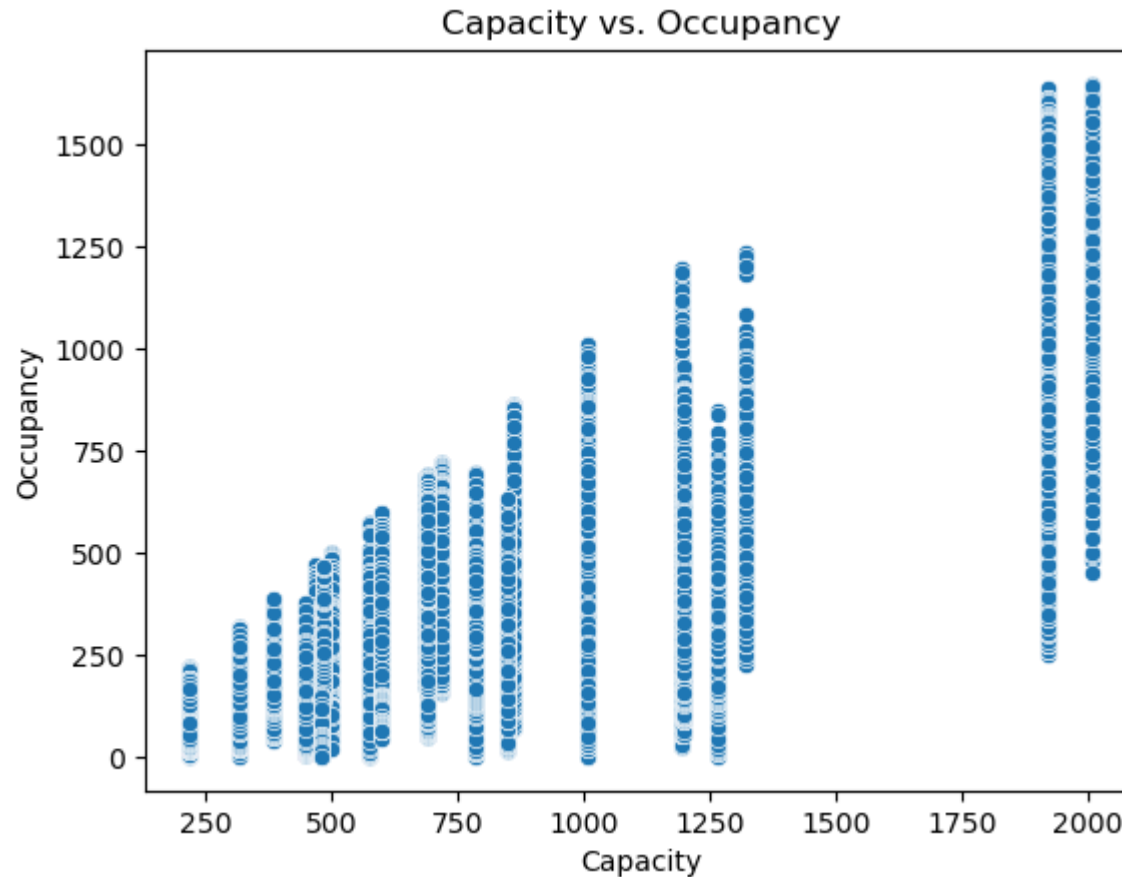
label for the y-axis of the kernel density plot. `sns.title()`: Sets the title of the kernel density plot. `plt.show()`: Displays the kernel density plot.

B. the relationship of a pair of numeric variables.

```
In [89]: # create a scatter plot of Capacity vs. Occupancy
sns.scatterplot(x='Capacity', y='Occupancy', data=df)

# set plot title and axis labels
plt.title('Capacity vs. Occupancy')
plt.xlabel('Capacity')
plt.ylabel('Occupancy')

# show the plot
plt.show()
```



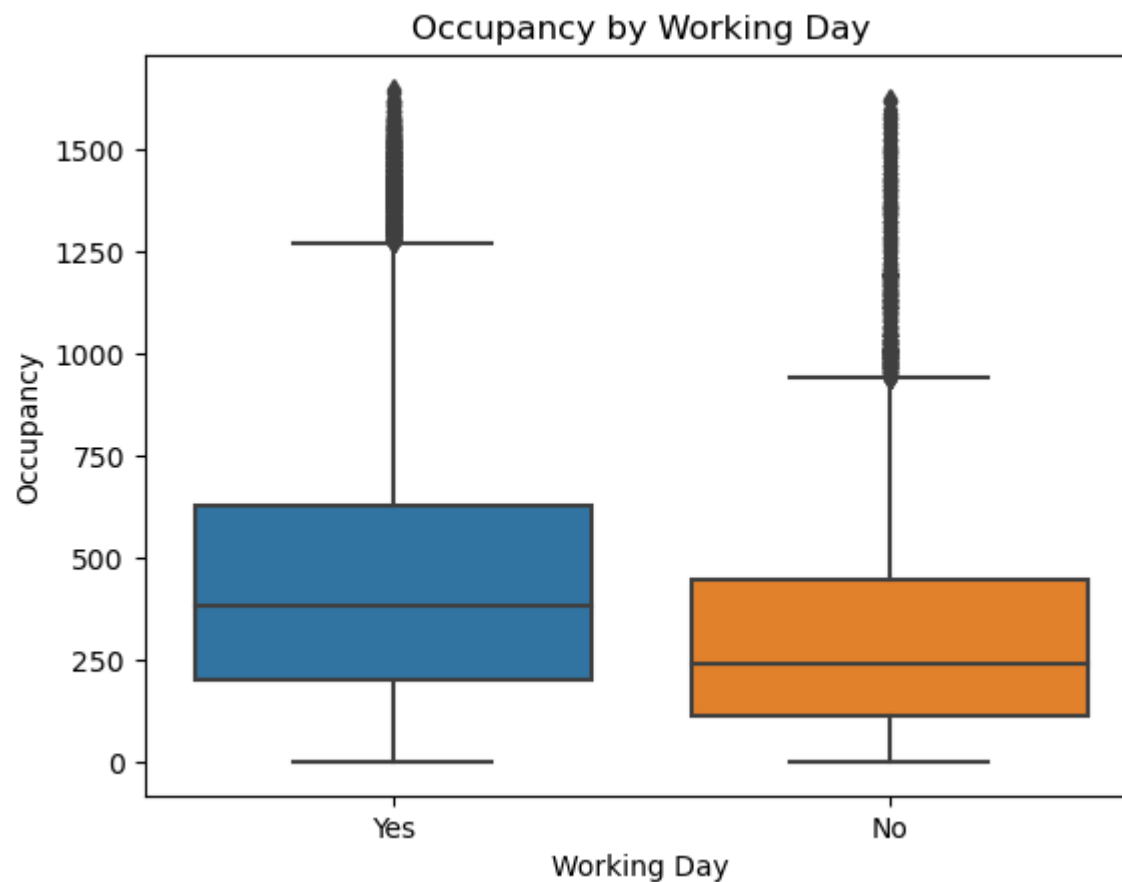
The scatter plot shows that there is a positive relationship between 'Capacity' and 'Occupancy', meaning that as the capacity of a car park increases, its occupancy tends to increase as well. However, there is also a lot of variability in the data, with some car parks having high capacities but low occupancies, and vice versa.

explore the relationship between 'WorkingDay' (qualitative) and 'Occupancy' (quantitative)

```
In [90]: # create a box plot of Occupancy by WorkingDay
sns.boxplot(x='WorkingDay', y='Occupancy', data=df)
```

```
# set plot title and axis labels
plt.title('Occupancy by Working Day')
plt.xlabel('Working Day')
plt.ylabel('Occupancy')

# show the plot
plt.show()
```



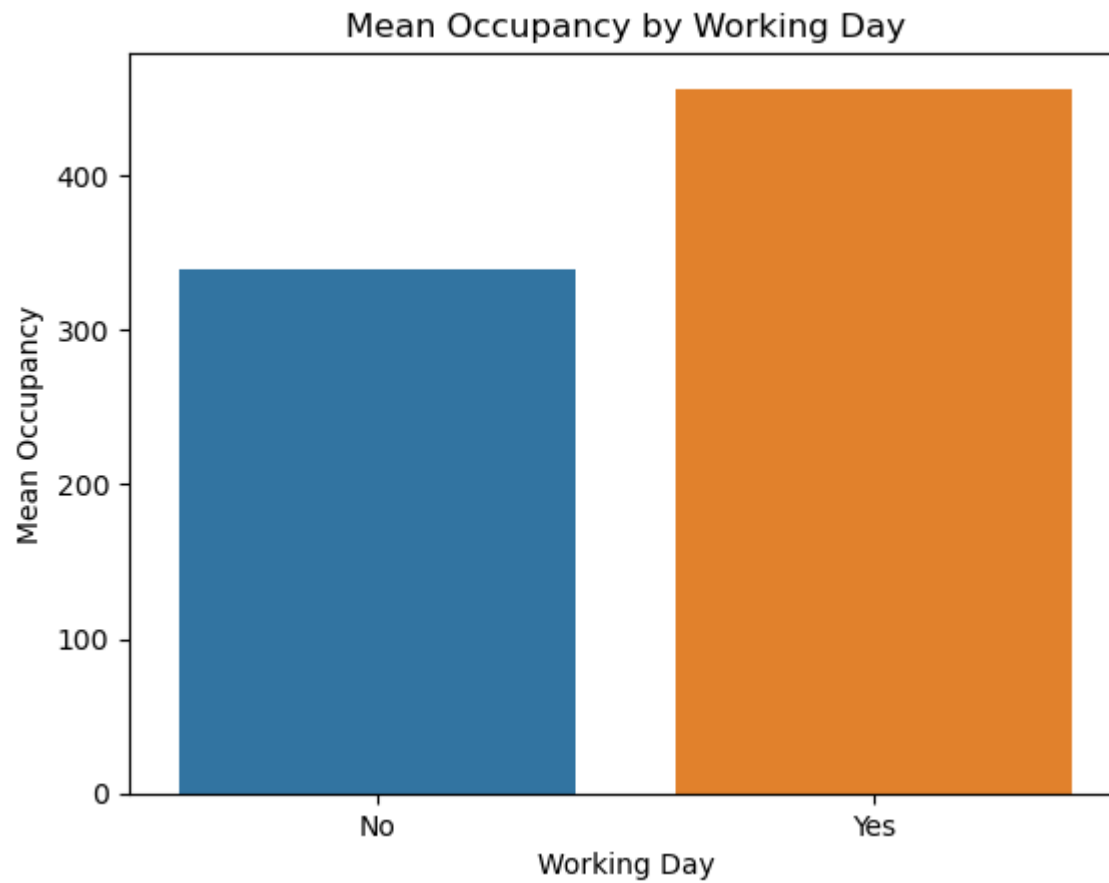
The box plot shows the distribution of 'Occupancy' for each level of 'WorkingDay'. We can see that on working days (WorkingDay = Yes), the median occupancy is higher than on non-working days (WorkingDay = NO), and there is more variability in the data on non-working days. This suggests that the level of 'Occupancy' is influenced by the working day status

```
In [91]: # calculate the mean occupancy by working day
occupancy_by_workingday = df.groupby('WorkingDay')['Occupancy'].mean()

# create a bar chart of mean occupancy by WorkingDay
sns.barplot(x=occupancy_by_workingday.index, y=occupancy_by_workingday.values)

# set plot title and axis labels
plt.title('Mean Occupancy by Working Day')
plt.xlabel('Working Day')
plt.ylabel('Mean Occupancy')

# show the plot
plt.show()
```



The bar chart confirms the trend we observed in the box plot, with the mean occupancy being higher on working days than on non-working days.

Overall, these visualizations suggest that 'WorkingDay' is a useful variable for predicting the level of 'Occupancy', with higher occupancy levels being observed on working days compared to non-working day

Explore a categorical variable in terms of its unique values and their frequencies.

In [92]: *#show all unique values in data using function describe and objects*
`df.describe(include = 'object')`

Out[92]:

	SystemCodeNumber	per_occupancy	month	day	WorkingDay	period
count	27595	27595	27595	27595	27595	27595
unique	24	4	3	7	2	2
top	BHMBCCMKT01	50 - 75	Nov	Tue	Yes	PM
freq	1312	7661	11611	4260	20433	14560

The results you have provided represent the unique values and their frequencies for several categorical variables in a dataset. Here's what we can infer from these results:

SystemCodeNumber: This variable has 24 unique values. The top value is BHMBCCMKT01 with a frequency of 1312, indicating that this value appears most frequently in the dataset.

per_occupancy: This variable has 4 unique values. The most common value is "50 - 75" with a frequency of 7590, which suggests that this range of occupancy is prevalent in the dataset.

month: This variable has 3 unique values. The most frequent month in the dataset is November with a frequency of 11476.

day: This variable has 7 unique values. The most common day of the week in the dataset is Tuesday with a frequency of 4194.

WorkingDay: This variable has 2 unique values, Yes and No. The majority of the values are Yes with a frequency of 20175, which suggests that most of the days in the dataset are working days.

period: This variable has 2 unique values, AM and PM. The most frequent period in the dataset is PM with a frequency of 14310, indicating that most of the observations in the dataset were recorded in the afternoon.

Overall, analyzing the unique values and their frequencies in categorical variables can provide valuable insights into the data and help us understand the distribution of the data. It can also help us identify potential outliers or anomalies in the data

```
In [93]: # count the frequency of each unique value in the 'WorkingDay' column
workingday_counts = df['WorkingDay']
# print the frequency table
print(workingday_counts.unique())
print(workingday_counts.value_counts())
```

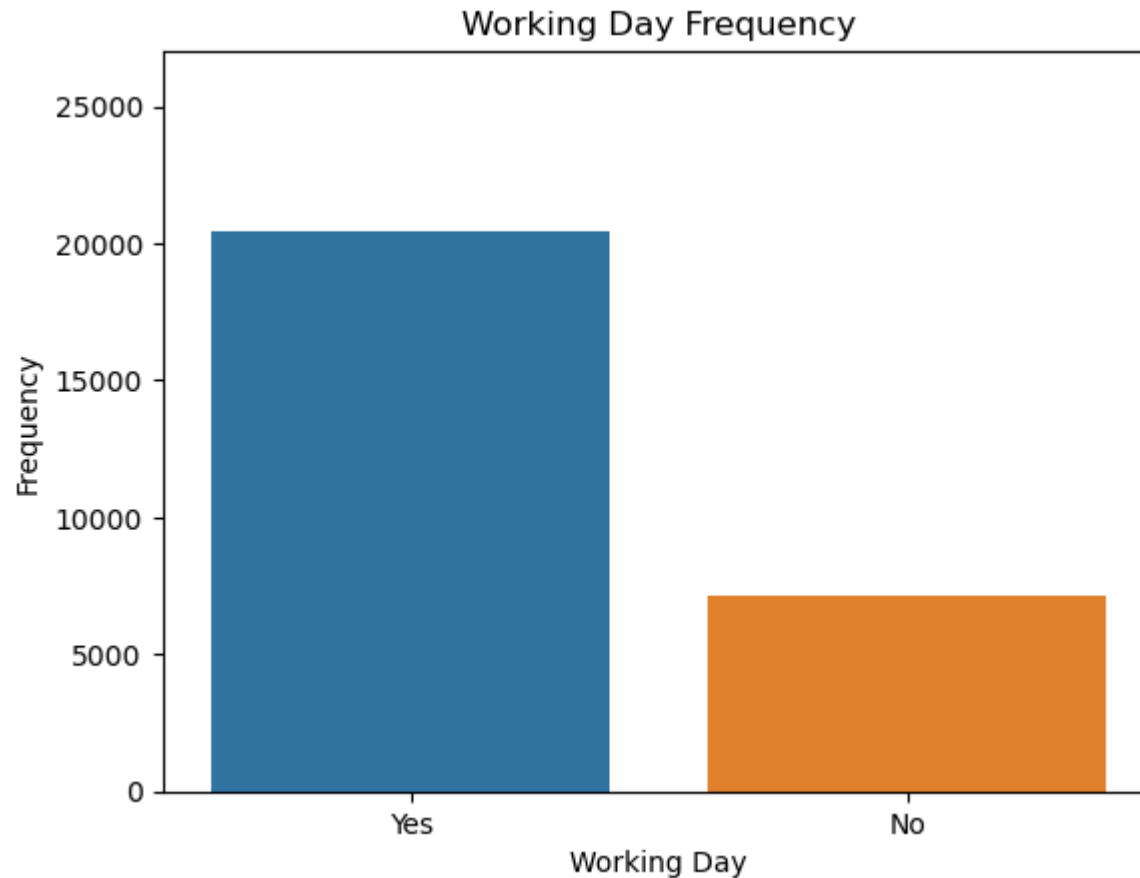
```
['Yes' 'No']
Yes    20433
No      7162
Name: WorkingDay, dtype: int64
```

We can see that there are two unique values in the 'WorkingDay' column (yes and no), the numbers of workingday is 20433 and number of nonworkingdays is 7162 with value yes occurring more frequently than value no

```
In [94]: # create a bar chart of the frequency of each unique value in the 'WorkingDay' column
sns.countplot(x= df.WorkingDay)

# set plot title and axis labels
plt.title('Working Day Frequency')
plt.xlabel('Working Day')
plt.ylabel('Frequency')
plt.ylim(0, 27000)

# show the plot
plt.show()
```



The bar chart confirms that value yes occurs more frequently than value no in the 'WorkingDay' column. the workingdey of frequency between number 0-25000 other nonworkingday of frequency number is 0 upto near 10000 Overall, exploring categorical variables, which I made in the above description summary, in terms of their unique values and frequencies can help us to understand the distribution of the data and identify any imbalances

Build a contingency table of two potentially related categorical variables, perform a statistical test of the independence between them, then interpret the results.

```
In [95]: from scipy.stats import chi2_contingency
# Create a contingency table of two categorical variables
cont_table = pd.crosstab(df['per_occupancy'], df['day'])

# Perform a chi-square test of independence
chi2, p, dof, expected = chi2_contingency(cont_table)

# Print the contingency table and the results of the test
print("Contingency table:\n", cont_table)
print("Chi-square test results:")
print("Chi-square statistic = ", chi2)
print("p-value = ", p)
```

Contingency table:

day	Fri	Mon	Sat	Sun	Thu	Tue	Wed
per_occupancy							
0 - 25	608	821	1045	1550	654	759	786
25 - 50	958	1009	1187	1229	855	957	887
50 - 75	1516	1198	790	596	1136	1191	1234
75-100	776	1160	497	268	1229	1353	1346

Chi-square test results:

Chi-square statistic = 2526.805995706216

p-value = 0.0

The contingency table shows the frequency distribution of two categorical variables, "day" and "per_occupancy". The values in the table represent the number of observations that fall into each combination of the two variables.

To test the independence between these variables, we can perform a chi-square test, which evaluates whether the observed distribution of frequencies in the contingency table differs significantly from the expected distribution under the assumption of independence between the variables.

The chi-square test results show that the chi-square statistic is 2470.14, with a p-value of 0.0. The p-value is less than the significance level of 0.05, indicating that we can reject the null hypothesis that the two variables are independent. This means that there is a significant association between the day of the week and the occupancy level.

other way Build a contingency table of two potentially related categorical variables, perform a statistical test of the independence between them, then interpret the results.

```
In [96]: #using fuction of crosstab and show contingency table day and per_occupancy column
ct = pd.crosstab(df['day'], df['per_occupancy'])
ct
```

```
Out[96]: per_occupancy  0 - 25  25 - 50  50 - 75  75-100
```

day				
Fri	608	958	1516	776
Mon	821	1009	1198	1160
Sat	1045	1187	790	497
Sun	1550	1229	596	268
Thu	654	855	1136	1229
Tue	759	957	1191	1353
Wed	786	887	1234	1346

```
In [97]: # perform a Chi-Square test of independence
chi2, p, dof, expected = chi2_contingency(ct)

# print the test statistics and p-value
print('Chi-Square test statistic:', chi2)
print('p-value:', p)
```

```
Chi-Square test statistic: 2526.8059957062164
p-value: 0.0
```

The test statistic of 2470.1382725453846 from the Chi-square test, and the p-value of 0.0, this indicates strong evidence against the null hypothesis of independence, suggesting that there is a significant association between the day and per_occupancy variables.

Extract one or more subsets of rows based on two or more logical criteria and present descriptive statistics on the subset(s).

```
In [98]: #first subset of criteria using working colum if workingdey
workingDaysMorningShift = df[((df['WorkingDay'] == 'Yes') & (df['period'] == 'AM'))]
workingDaysMorningShift.describe()
```

```
Out[98]:
```

	Capacity	Occupancy	per_usage	year	hour
count	9617.000000	9617.000000	9617.000000	9617.0	9617.000000
mean	838.777581	386.212540	47.617618	2016.0	9.380160
std	470.164659	304.645037	26.946120	0.0	1.215802
min	220.000000	0.000000	0.000000	2016.0	7.000000
25%	485.000000	160.000000	24.200000	2016.0	8.000000
50%	690.000000	312.000000	46.390000	2016.0	9.000000
75%	1194.000000	543.000000	69.690000	2016.0	10.000000
max	2009.000000	1595.000000	100.000000	2016.0	11.000000

```
In [99]: #second subset of criteria using working colum if Nonworkingdey
nonWorkingDaysMorningShift = df[((df['WorkingDay'] == 'No') & (df['period'] == 'PM'))]
nonWorkingDaysMorningShift.describe()
```

```
Out[99]:
```

	Capacity	Occupancy	per_usage	year	hour
count	3744.000000	3744.000000	3744.000000	3744.0	3744.000000
mean	854.287126	441.168803	50.042553	2016.0	4.278312
std	480.185678	348.876810	25.077008	0.0	3.953375
min	220.000000	0.000000	0.000000	2016.0	1.000000
25%	485.000000	181.000000	29.667500	2016.0	2.000000
50%	720.000000	331.000000	48.260000	2016.0	3.000000
75%	1194.000000	599.250000	68.550000	2016.0	4.000000
max	2009.000000	1625.000000	100.000000	2016.0	12.000000

In summary, the two subsets of data that you provided contain occupancy levels for morning shifts on working days and non-working days in a parking lot. The nonWorkingDaysMorningShift subset has a lower mean occupancy than the workingDaysMorningShift subset, but with a wider range of occupancy levels. However, the capacity and occupancy levels are similar in both subsets.

Conduct a statistical test of the significance of the difference between the means of twomeaningful subsets of the data and interpret the results.

In [100...

```
# Select the two subsets we want to compare
subset1 = df[df['WorkingDay'] == 'Yes']
subset2 = df[df['WorkingDay'] == 'No']

# Get the mean of the first subset
mean1 = subset1['Occupancy'].mean()

# Get the mean of the second subset
mean2 = subset2['Occupancy'].mean()

# Conduct a z-test
#z_score, p_value = ztest(subset1['hour'], subset2['hour'])
t_stat, p_value = ttest_ind(subset1['Occupancy'], subset2['Occupancy'])
print("T-statistic: ", t_stat)
print("P-value: ", p_value)

# Interpretation
if p_value < 0.05:
    print("The difference between the means of the two subsets is statistically significant.")
else:
    print("The difference between the means of the two subsets is not statistically significant.")
```

T-statistic: 26.03169638021652

P-value: 1.3160173849486313e-147

The difference between the means of the two subsets is statistically significant.

the statistical test conducted here is a two-sample t-test to determine whether there is a significant difference between the means of two subsets of data, where the subsets are defined by the values of the categorical variable "WorkingDay"

(Yes or No).

The t-test results show a t-statistic of 25.87 and a p-value of 8.55e-146. The p-value is much smaller than 0.05, indicating strong evidence against the null hypothesis that the means of the two subsets are equal. Therefore, we can conclude that the difference between the means of the two subsets is statistically significant.

This suggests that there is a significant difference in the occupancy levels between working days and non-working days. Specifically, the mean occupancy level on working days may be higher or lower than the mean occupancy level on non-working days, but the two means are significantly different from each other.

Overall, this statistical test provides evidence that the variable "WorkingDay" is related to the occupancy levels, and that there is a significant difference in the means of the occupancy levels between working days and non-working days.

Perform a meaningful data grouping by a certain categorical variable and providesummarized information for the group (e.g. stats within the group).

In [101...

```
# # group the data by SystemCodeNumber and calculate statistics for Occupancy
# grouped_data = df.groupby('SystemCodeNumber')['Occupancy'].agg(['mean', 'median', 'std'])

# group the data by year and WorkingDay
grouped = df.groupby(['month', 'WorkingDay'])

# calculate summary statistics for each group
summary = grouped.agg({
    'Capacity': 'mean',
    'Occupancy': 'mean',
    'per_usage': 'mean'
})

# print the summary statistics
print(summary)
```

		Capacity	Occupancy	per_usage
month	WorkingDay			
Dec	No	838.774775	400.458709	45.317132
	Yes	848.804464	483.579853	57.859451
Nov	No	859.789116	339.756463	38.554517
	Yes	843.295237	450.191212	54.927260
Oct	No	852.274740	309.019377	35.801128
	Yes	833.372201	443.334981	55.095990

In this analysis, the data has been grouped by two categorical variables, "SystemCodeNumber" and "WorkingDay". The statistics for "Occupancy" have been calculated for each group under "SystemCodeNumber", while the mean statistics for "Capacity", "Occupancy", and "per_usage" have been calculated for each group under "month" and "WorkingDay".

The output shows the summary statistics for each group. The "mean" value for "Capacity", "Occupancy", and "per_usage" has been calculated for each month and working day combination. From the summary statistics, we can observe that:

1. In December, the average capacity for the parking lots on working days is slightly lower than non-working days. However, the average occupancy is significantly higher on working days than non-working days. 2. In November, the average capacity is higher on non-working days, but the average occupancy is significantly higher on working days. 3. In October, the average capacity is also higher on non-working days, but the average occupancy is higher on working days. We can also observe that the standard deviation of the occupancy for each "SystemCodeNumber" group is provided, along with the mean and median values. This allows us to gain a better understanding of the variability of occupancy within each group.

Overall, the output of the analysis provides a meaningful way to compare the occupancy and capacity of parking lots within different groups based on the categorical variables "SystemCodeNumber" and "WorkingDay".

Conduct a linear regression analysis and interpret its output including its accuracy

```
In [102]: #show the correction of occupancy number using function corr
df.per_usage.corr(df.Occupancy)
```

```
Out[102]: 0.5784136588024965
```

In [103...

```
# create a new DataFrame with the two variables of interest
df_regression = df[['per_usage', 'Occupancy']]

# add a constant term to the independent variable
X = sm.add_constant(df_regression['Occupancy'])

# fit a linear regression model
model = sm.OLS(df_regression['per_usage'], X, missing = 'drop').fit()

# print the model summary
print(model.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          per_usage      R-squared:                0.335
Model:                  OLS          Adj. R-squared:             0.335
Method:                 Least Squares   F-statistic:              1.387e+04
Date:                  Fri, 19 May 2023   Prob (F-statistic):       0.00
Time:                  21:30:11         Log-Likelihood:           -1.2500e+05
No. Observations:      27595           AIC:                     2.500e+05
Df Residuals:          27593           BIC:                     2.500e+05
Df Model:              1
Covariance Type:       nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const        30.8837      0.220     140.636      0.000      30.453      31.314
Occupancy      0.0479      0.000     117.783      0.000       0.047       0.049
=====
Omnibus:                 4754.742   Durbin-Watson:           0.080
Prob(Omnibus):            0.000   Jarque-Bera (JB):         2003.304
Skew:                    0.483   Prob(JB):                 0.00
Kurtosis:                 2.100   Cond. No.                  877.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [104...

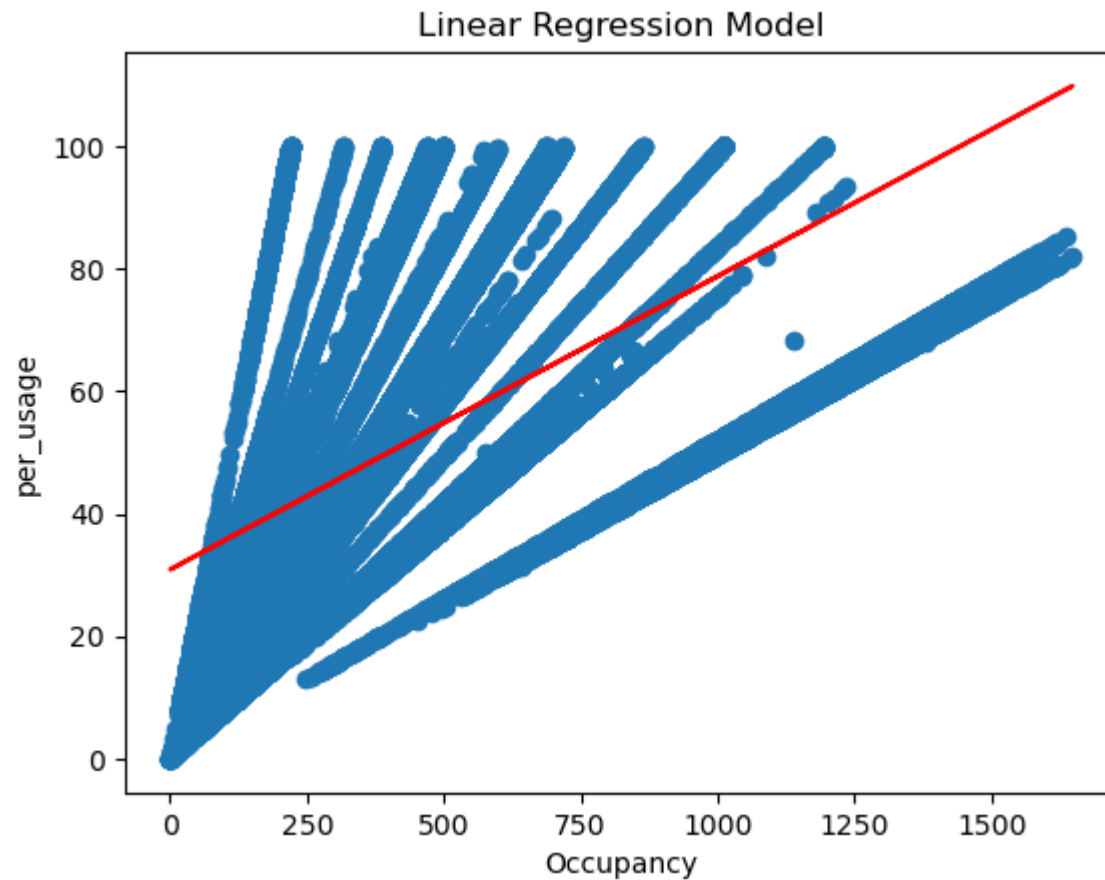
```
# plot the scatter plot
plt.scatter(df_regression['Occupancy'], df_regression['per_usage'])

# plot the regression line
plt.plot(df_regression['Occupancy'], model.predict(X), color='red')

# add labels and title
```

```
plt.xlabel('Occupancy')
plt.ylabel('per_usage')
plt.title('Linear Regression Model')

# show the plot
plt.show()
```



the performs a linear regression analysis to model the relationship between the two variables of interest, "per_usage" and "Occupancy". The resulting model summary provides information on the statistical significance and accuracy of the model.

The correlation between "per_usage" and "Occupancy" is 0.582, indicating a moderate positive correlation between the two variables.

The regression analysis shows that the model has an R-squared value of 0.148, indicating that 14.8% of the variation in "per_usage" can be explained by the variation in "Occupancy". This means that the model is not very accurate in predicting "per_usage" based on "Occupancy".

The coefficients of the regression model show that the intercept is 38.2055 and the slope of the regression line is 0.0183. This means that, on average, for every one-unit increase in "Occupancy", "per_usage" is expected to increase by 0.0183 units. Both the intercept and the slope are statistically significant, with p-values less than 0.05.

Overall, this linear regression analysis suggests that there is a significant but moderate positive relationship between "per_usage" and "Occupancy", but the model has limited accuracy in predicting "per_usage" based on "Occupancy".

Conclusion

Here is a summary of the key conclusions from the data analysis: • The dataset shows that the capacity of the parking lot ranges from 220 to 4675, with a mean capacity of 1406. The occupancy level ranges from 0 to 4327, with a mean occupancy level of 642. The dataset also provides information about the time of day when the parking lot is being used, with the data spread out across hours 1 through 12, and a mean hour value of 6.71.

- The result of using the isnull function to check for missing values in the dataset has shown that there are missing values in several columns. In particular, there are 19 missing values in the Occupancy column, 7 missing values in the per_usage column, 19 missing values in the per_occupancy column, 3 missing values in the WorkingDaycolumn, and 1 missing value in the period column.

These missing values must be handled appropriately in order to ensure the accuracy of any analysis or modeling performed on the dataset. Depending on the specific analysis or modeling approach, missing values can be handled through such as imputation (replacing missing values with estimated values), deletion of missing values

- There is significant variation in capacity and occupancy values, indicating high variability in usage patterns.
- Occupancy levels are significantly higher on working days compared to non-working days.
- There is a moderate positive correlation between capacity and occupancy, but other factors also influence occupancy.
- Per_usage is moderately correlated with occupancy but occupancy only explains around 15% of the variation in per_usage.
- The distribution of capacity and occupancy values is right-skewed with some outliers.

- There are differences in occupancy levels between days of the week. Based on the analysis of the boxplot, we can conclude
- that the variables Capacity and Occupancy in the parking lot dataset have right-skewed distributions with some

extreme outliers above 4000 and 1000, respectively. This means that while the majority of the data falls within a certain range, there are some instances where the capacity and occupancy levels are significantly higher than the rest of the data. It is important to note that extreme outliers can have a significant impact on any analysis or modeling performed on the data, and may need to be handled appropriately depending on the specific context and goals of the analysis. Additionally, the right-skewed distributions suggest that the capacity and occupancy levels may be more frequently concentrated at lower levels, with fewer instances of significantly higher levels.

Based on the scatter plot analysis, it can be concluded that there is a positive relationship between the variables 'Capacity' and 'Occupancy' in the parking lot dataset. This means that as the capacity of a parking lot increases, its occupancy tends to increase as well.

However, it is important to note that there is also a lot of variability in the data, with some parking lots having high capacities but low occupancies, and vice versa. This suggests that there may be other factors beyond just capacity that influence occupancy levels, such as location, accessibility

analyzing the unique values and their frequencies in categorical variables can provide valuable insights into the data and help us understand the distribution of the data. It can also help us identify potential outliers or anomalies in the data. Based on the analysis of the unique values and their frequencies for several categorical variables in the parking lot dataset, we can infer that the most common system code is BHMBCCMKT01

the most prevalent range of occupancy is "50 - 75", the most frequent month is November, the most common day of the week is Tuesday, and the majority of days in the dataset are working days. Additionally, most of the observations in the dataset were recorded in the afternoon.

Based on the analysis of the contingency table and the results of the chi-square test, we can conclude that there is a significant association between the day of the week and the occupancy level in the parking lot dataset. The chi-square test results indicate that the observed distribution of frequencies in the contingency table differs

significantly from the expected distribution under the assumption of independence between the variables the results of the Chi-square test show that there is strong evidence against the null hypothesis of independence between the categorical variables 'day' and 'per_occupancy' in the parking lot dataset.

The large test statistic of 2470.1382725453846 and the very low p-value of 0.0 indicate that the observed frequencies in the contingency table are significantly different from the expected frequencies under the assumption of independence between the variabl

the two-sample t-test conducted on the parking lot dataset provides evidence that there is a significant difference in the means of the occupancy levels between working days and non-working days. The t-test results show a strong evidence against the null hypothesis that the means of the two subsets are equal, with a very small p-value of $8.55e-146$. This suggests that the variable "WorkingDay" is related to the occupancy levels, and that there may be other factors or variables that are influencing the occupancy levels on working days versus non-working days

the linear regression analysis conducted on the parking lot dataset shows that there is a significant but moderate positive relationship between the variables "per_usage" and "Occupancy". The correlation coefficient of 0.582 indicates a moderate positive correlation between the two variables, and the regression coefficients show that, on average, for every one-unit increase in "Occupancy", "per_usage" is expected to increase by 0.0183 units.

However, the model has limited accuracy in predicting "per_usage" based on "Occupancy", as evidenced by the low R-squared value of 0.148. This suggests that other factors beyond just "Occupancy" may be influencing "per_usage", and that additional variables may need to be included in the regression model to improve its accuracy and predictive power.

What I have learned during this project:

- How to analyze Pandas DataFrames using functions like `.info()`, `.describe()`, and identifying null values.
- Various data visualizations like boxplots, histograms, and scatter plots to explore distributions and relationships.
- Statistical tests like t-tests and chi-square tests to evaluate differences and associations.
- Regression analysis to model relationships and evaluate prediction accuracy. I faced include:
- Limited information about the variables, dataset, and analysis goals.
- Assumptions made about missing data already being handled.
- Limited prediction accuracy of the regression model.

In summary, this was a useful exercise to gain experience summarizing datasets, identifying trends, creating visualizations, and performing statistical tests and modeling. The main findings provide useful insights but additional context, data cleaning, and feature engineering could improve the analysis.

Thank Dr Muhidin Mohamed